

```
In [50]: import pandas as pd
import numpy as np

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    make_scorer
)
from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV, Pre
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier
```

```
In [36]: train_df = pd.read_csv("../data/processed/train.csv")
val_df = pd.read_csv("../data/processed/val.csv")
test_df = pd.read_csv("../data/processed/test.csv")
```

```
In [37]: X_train = train_df.drop(columns=["disease"])
y_train = train_df["disease"]

X_val = val_df.drop(columns=["disease"])
y_val = val_df["disease"]

X_test = test_df.drop(columns=["disease"])
y_test = test_df["disease"]
```

```
In [38]: label_encoder = LabelEncoder()

all_y = pd.concat([y_train, y_val, y_test], axis=0)
label_encoder.fit(all_y)

y_train_enc = label_encoder.transform(y_train)
y_val_enc = label_encoder.transform(y_val)
y_test_enc = label_encoder.transform(y_test)
```

```
In [39]: print(X_train.shape, X_val.shape, X_test.shape)
print(y_train.shape, y_val.shape, y_test.shape)
print(y_train_enc.shape, y_val_enc.shape, y_test_enc.shape)

(69979, 175) (14995, 175) (14996, 175)
(69979,) (14995,) (14996,)
(69979,) (14995,) (14996,)
```

```
In [40]: sample_weights = compute_sample_weight(
    class_weight="balanced",
    y=y_train_enc
)

print("Min weight:", sample_weights.min())
print("Max weight:", sample_weights.max())
```

```
Min weight: 0.12687491274732168
Max weight: 236.41554054054055
```

```
In [41]: X_train_val = pd.concat([X_train, X_val], axis=0).reset_index(drop=True)
y_train_val = np.concatenate([y_train_enc, y_val_enc])
```

```

train_weights = compute_sample_weight(class_weight="balanced", y=y_train_enc)
val_weights = np.ones(len(y_val_enc))
sample_weights_train_val = np.concatenate([train_weights, val_weights])

test_fold = np.concatenate([
    -1 * np.ones(len(X_train), dtype=int),    # training fold
    np.zeros(len(X_val), dtype=int)          # validation fold
])

ps = PredefinedSplit(test_fold=test_fold)

print(X_train_val.shape, y_train_val.shape)

```

(84974, 175) (84974,)

```

In [42]: xgb = XGBClassifier(
    objective="multi:softprob",
    num_class=len(label_encoder.classes_),
    eval_metric="mlogloss",
    random_state=42,
    tree_method="hist"
)

param_dist = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [3, 4, 5, 6, 8],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "subsample": [0.7, 0.8, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.8, 0.9, 1.0],
    "min_child_weight": [1, 3, 5],
    "gamma": [0, 0.1, 0.3],
    "reg_alpha": [0, 0.01, 0.1],
    "reg_lambda": [1, 1.5, 2]
}

```

```

In [43]: search = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist,
    n_iter=20,
    scoring="f1_macro",
    cv=ps,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

search.fit(X_train_val, y_train_val, sample_weight=sample_weights_train_val)

print("Best validation Macro F1:", search.best_score_)
print("Best params:", search.best_params_)

```

Fitting 1 folds for each of 20 candidates, totalling 20 fits

[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.1, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 3.0min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.7; total time= 3.0min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.05, max_depth=5, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.9; total time= 3.5min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.05, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 2.8min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=0.9; total time= 5.8min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min_child_weight=5, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=0.8; total time= 6.2min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.05, max_depth=4, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1.5, subsample=0.9; total time= 6.4min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 6.5min

[CV] END colsample_bytree=0.9, gamma=0.3, learning_rate=0.2, max_depth=5, min_child_weight=5, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time= 6.8min

[CV] END colsample_bytree=0.8, gamma=0.3, learning_rate=0.01, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0.1, reg_lambda=1.5, subsample=1.0; total time= 6.8min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=5, min_child_weight=3, n_estimators=100, reg_alpha=0, reg_lambda=1, subsample=0.8; total time= 3.4min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=8, min_child_weight=3, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=1.0; total time= 7.8min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=4, min_child_weight=3, n_estimators=300, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 8.4min

[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=3, n_estimators=300, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 9.1min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=8, min_child_weight=3, n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=0.8; total time=10.2min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=3, min_child_weight=3, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=10.8min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time=11.8min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, min_child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsample=0.8; total time= 9.5min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=6, min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=1.0; total time=13.1min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=5, min_child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsample=1.0; total time= 9.6min

Best validation Macro F1: 0.4576869337207617

Best params: {'subsample': 1.0, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 500, 'min_child_weight': 1, 'max_depth': 6, 'learning_rate': 0.2, 'gamma': 0, 'colsample_bytree': 0.9}

```
In [45]: best_model = search.best_estimator_
y_pred = best_model.predict(X_test)

results = {
    "Accuracy": accuracy_score(y_test_enc, y_pred),
    "Precision_Macro": precision_score(y_test_enc, y_pred, average="macro",
    "Recall_Macro": recall_score(y_test_enc, y_pred, average="macro", zero_div
    "F1_Macro": f1_score(y_test_enc, y_pred, average="macro", zero_division=
    "Precision_Weighted": precision_score(y_test_enc, y_pred, average="weigh
    "Recall_Weighted": recall_score(y_test_enc, y_pred, average="weighted",
    "F1_Weighted": f1_score(y_test_enc, y_pred, average="weighted", zero_div
}

pd.DataFrame([results])
```

```
Out[45]:
```

	Accuracy	Precision_Macro	Recall_Macro	F1_Macro	Precision_Weighted	Recall_W
0	0.71319	0.471098	0.457734	0.458393	0.713573	

```
In [48]: param_dist = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [3, 4, 5, 6, 8],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "subsample": [0.7, 0.8, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.8, 0.9, 1.0],
    "min_child_weight": [1, 3, 5],
    "gamma": [0, 0.1, 0.3],
    "reg_alpha": [0, 0.01, 0.1],
    "reg_lambda": [1, 1.5, 2]
}
```

```
In [51]: cv = StratifiedKFold(
    n_splits=3,
    shuffle=True,
    random_state=42
)

f1_macro_scorer = make_scorer(f1_score, average="macro")
```

```
In [52]: search = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist,
    n_iter=20,
    scoring=f1_macro_scorer,
    cv=cv,
    verbose=2,
    random_state=42,
    n_jobs=-1,
    refit=True
)
```

```
In [53]: search.fit(
    X_train,
```

```
y_train_enc,  
sample_weight=sample_weights  
)
```

Fitting 3 folds for each of 20 candidates, totalling 60 fits

[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.1, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 2.0min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.05, max_depth=5, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.9; total time= 2.3min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.05, max_depth=5, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.9; total time= 2.3min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.05, max_depth=5, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.9; total time= 2.3min

[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.1, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 2.0min

[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.1, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 2.0min

[CV] END colsample_bytree=0.9, gamma=0.3, learning_rate=0.2, max_depth=5, min_child_weight=5, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time= 4.6min

[CV] END colsample_bytree=0.9, gamma=0.3, learning_rate=0.2, max_depth=5, min_child_weight=5, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time= 4.6min

[CV] END colsample_bytree=0.9, gamma=0.3, learning_rate=0.2, max_depth=5, min_child_weight=5, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time= 4.6min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=4, min_child_weight=3, n_estimators=300, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 5.9min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=4, min_child_weight=3, n_estimators=300, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 5.9min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=4, min_child_weight=3, n_estimators=300, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 6.2min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.05, max_depth=4, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1.5, subsample=0.9; total time= 4.3min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.05, max_depth=4, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1.5, subsample=0.9; total time= 4.4min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.05, max_depth=4, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1.5, subsample=0.9; total time= 4.3min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=8, min_child_weight=3, n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=0.8; total time= 8.3min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=8, min_child_weight=3, n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=0.8; total time= 8.4min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=8, min_child_weight=3, n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=0.8; total time= 8.5min

[CV] END colsample_bytree=0.8, gamma=0.3, learning_rate=0.01, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0.1, reg_lambda=1.5, subsample=1.0; total time= 4.5min

[CV] END colsample_bytree=0.8, gamma=0.3, learning_rate=0.01, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0.1, reg_lambda=1.5, subsample=1.0; total time= 4.5min

[CV] END colsample_bytree=0.8, gamma=0.3, learning_rate=0.01, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0.1, reg_lambda=1.5, subsample=1.0; total time= 4.6min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time=10.6min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time=10.7min

[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2, subsample=0.7; total time=10.8min

[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=3, n_estimators=300, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 6.7min

[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=3, n_estimators=300, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 6.7min

[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.01, max_depth=5, min_child_weight=3, n_estimators=300, reg_alpha=0.01, reg_lambda=1.5, subsample=0.9; total time= 6.8min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=8, min_child_weight=3, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=1.0; total time= 5.6min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=8, min_child_weight=3, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=1.0; total time= 5.7min

[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=8, min_child_weight=3, n_estimators=200, reg_alpha=0.1, reg_lambda=2, subsample=1.0; total time= 5.7min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=0.9; total time= 4.0min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 4.5min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 4.5min

[CV] END colsample_bytree=0.7, gamma=0.1, learning_rate=0.2, max_depth=5, min_child_weight=1, n_estimators=200, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 4.6min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.7; total time= 2.2min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=0.9; total time= 4.1min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.7; total time= 2.2min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, min_child_weight=1, n_estimators=100, reg_alpha=0.01, reg_lambda=2, subsample=0.7; total time= 2.2min

[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.2, max_depth=3, min_child_weight=1, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=0.9; total time= 4.2min

[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.05, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 2.1min

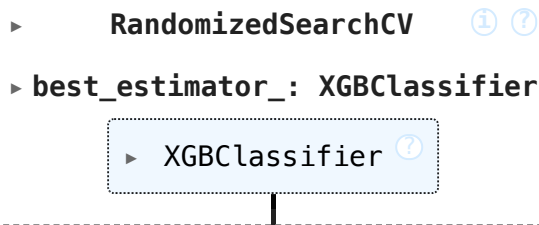
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.05, max_depth=3, min_child_weight=5, n_estimators=100, reg_alpha=0.1, reg_lambda=1, subsample=1.0; total time= 2.1min

```

in_child_weight=5, n_estimators=100, reg_alpha=0.1, reg_lambda=1, subsample=
1.0; total time= 2.1min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.05, max_depth=3, m
in_child_weight=5, n_estimators=100, reg_alpha=0.1, reg_lambda=1, subsample=
1.0; total time= 2.2min
[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min
_child_weight=5, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=
0.8; total time= 4.5min
[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min
_child_weight=5, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=
0.8; total time= 4.5min
[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.01, max_depth=4, min
_child_weight=5, n_estimators=200, reg_alpha=0.01, reg_lambda=1, subsample=
0.8; total time= 4.4min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=5, mi
n_child_weight=3, n_estimators=100, reg_alpha=0, reg_lambda=1, subsample=0.
8; total time= 2.4min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=5, mi
n_child_weight=3, n_estimators=100, reg_alpha=0, reg_lambda=1, subsample=0.
8; total time= 2.4min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=5, mi
n_child_weight=3, n_estimators=100, reg_alpha=0, reg_lambda=1, subsample=0.
8; total time= 2.4min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=3, mi
n_child_weight=3, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.
8; total time= 9.2min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=3, mi
n_child_weight=3, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.
8; total time= 9.2min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.2, max_depth=3, mi
n_child_weight=3, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.
8; total time= 9.9min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, mi
n_child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsampl
e=0.8; total time= 8.3min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, mi
n_child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsampl
e=0.8; total time= 8.3min
[CV] END colsample_bytree=0.9, gamma=0.1, learning_rate=0.1, max_depth=3, mi
n_child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsampl
e=0.8; total time= 8.2min
[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=6, min_
child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=1.0;
total time= 9.8min
[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=6, min_
child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=1.0;
total time=10.0min
[CV] END colsample_bytree=0.9, gamma=0, learning_rate=0.2, max_depth=6, min_
child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=1.0;
total time= 9.7min
[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=5, min_
child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsample=
1.0; total time= 8.3min
[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=5, min_
child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsample=
1.0; total time= 7.7min
[CV] END colsample_bytree=0.7, gamma=0, learning_rate=0.1, max_depth=5, min_
child_weight=1, n_estimators=500, reg_alpha=0.01, reg_lambda=1.5, subsample=
1.0; total time= 8.0min

```


Out [53]:



```
In [54]: print("Best CV Macro F1:", search.best_score_)
print("Best Parameters:")
print(search.best_params_)
```

Best CV Macro F1: 0.43318632686222475

Best Parameters:

```
{'subsample': 0.8, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 500, 'min_child_weight': 3, 'max_depth': 3, 'learning_rate': 0.2, 'gamma': 0.1, 'colsample_bytree': 0.9}
```

```
In [55]: best_model = search.best_estimator_
```

```
In [56]: y_pred_val = best_model.predict(X_val)
```

```
val_accuracy = accuracy_score(y_val_enc, y_pred_val)
val_precision_macro = precision_score(y_val_enc, y_pred_val, average="macro")
val_recall_macro = recall_score(y_val_enc, y_pred_val, average="macro", zero_division=0)
val_f1_macro = f1_score(y_val_enc, y_pred_val, average="macro", zero_division=0)

val_precision_weighted = precision_score(y_val_enc, y_pred_val, average="weighted")
val_recall_weighted = recall_score(y_val_enc, y_pred_val, average="weighted", zero_division=0)
val_f1_weighted = f1_score(y_val_enc, y_pred_val, average="weighted", zero_division=0)

print("Validation Accuracy:", val_accuracy)
print("Validation Precision Macro:", val_precision_macro)
print("Validation Recall Macro:", val_recall_macro)
print("Validation F1 Macro:", val_f1_macro)
print("Validation Precision Weighted:", val_precision_weighted)
print("Validation Recall Weighted:", val_recall_weighted)
print("Validation F1 Weighted:", val_f1_weighted)
```

Validation Accuracy: 0.6568189396465488

Validation Precision Macro: 0.4260964388093152

Validation Recall Macro: 0.5059806564409937

Validation F1 Macro: 0.44238585586898704

Validation Precision Weighted: 0.7076958740652625

Validation Recall Weighted: 0.6568189396465488

Validation F1 Weighted: 0.6656582746795167

In []: